

program counter (line num)	call stack	micro queue	promises	macro queue	event reg	console output
5	[main()]	[]	{promise@5}	[]	{}	
8	[main()]	[]	{promise@5:rs, promise@8}	[]	{}	
22	[main()]	[]	{promise@5:rs, promise@8:rs}	[]	{ ev1.run:function@22 }	
26	[main()]	[]	{promise@5:rs, promise@8:rs}	[]	{ ev1.run:function@22, ev2.run:function@26 }	
30	[main()]	[]	{promise@5:rs, promise@8:rs}	[function@26(0)]	{ ev1.run:function@22, ev2.run:function@26 }	
eof	[]	[]	{promise@5:rs, promise@8:rs}	[function@26(0)]	{ ev1.run:function@22, ev2.run:function@26 }	
27	[function@26(0)]	[]	{promise@5:rs, promise@8:rs}	[]	{ ev1.run:function@22, ev2.run:function@26 }	`data 0 received by ev1`
28	[function@26(0)]	[function@28(0)]	{promise@5:rs, promise@8:rs, promise@28}	[]	{ ev1.run:function@22, ev2.run:function@26 }	
16 -> 22	[function@28(0), foo(0)]	[]	{promise@5:rs, promise@8:rs, promise@28}	[function@22(1)]	{ ev1.run:function@22, ev2.run:function@26 }	
23	[function@22(1)]	[]	{promise@5:rs, promise@8:rs, promise@28}	[]	{ ev1.run:function@22, ev2.run:function@26 }	`data 1 received by ev1`
24	[function@22(1)]	[function@24(1)]	{promise@5:rs, promise@8:rs, promise@28, promise@24}	[]	{ ev1.run:function@22, ev2.run:function@26 }	
18	[function@24(1), foo(1)]	[]	{promise@5:rs, promise@8:rs, promise@28, promise@24}	[function@26(2)]	{ ev1.run:function@22, ev2.run:function@26 }	
26	[function@26(2)]	[]	{promise@5:rs, promise@8:rs, promise@28, promise@24}	[]	{ ev1.run:function@22, ev2.run:function@26 }	

Lines 27 & 28: The event loop picks up the macrotask [function@26(0)]. It prints to the console, and then promise1.then() queues its callback function@28(0) to the microtask queue. The .then() call returns a new pending promise (promise@28).

Lines 16 -> 22: The macrotask finishes, so the event loop clears the microtask queue. It runs function@28(0) which calls foo(0). Inside foo(0), x is 0. Because $0 \% 2 == 0$ is true, it proceeds to line 16 and executes ev1.emit('run', 1). This synchronously triggers the ev1 listener at line 22, pushing function@22(1) to the macrotask queue.

Lines 23 & 24: The stack clears, and the event loop starts the next cycle by picking up the [function@22(1)] macrotask. It logs the new data to the console, and promise2.then() queues a new microtask function@24(1).

Lines 18 & 26: The event loop clears the micro queue, executing foo(1). Because $1 \% 2 == 0$ is false, the execution hits the else block at line 18. It calls ev2.emit('run', 2), triggering the listener at line 26 synchronously, which pushes function@26(2) to the macrotask queue.

Cycle Reset: The call stack empties, leaving [function@26(2)] in the macrotask queue. This is the exact same state as eof, completing one full cycle before it repeats until $x > 10$ and the promise is resolved. At this point, there would be nothing in the macro and micro queues so the process will terminate.